

@tetherto/wdk-protocol-fiat-transak

Note: This package is in beta. Please test in a dev setup first.

Integrate the Transak widget for buying (on-ramp) and selling (off-ramp) crypto. Use it to generate widget URLs, get buy and sell quotes, and look up supported currencies, countries, and order details. Works in both frontend and backend code.

About WDK

This is part of WDK (Wallet Development Kit). WDK helps you build safe, non-custody wallets. Read more at <https://docs.wallet.tether.io>.

Features

- Generate a widget URL to buy crypto (on-ramp)
- Generate a widget URL to sell crypto (off-ramp)
- Get buy and sell quotes
- List supported currencies and countries
- Look up order (transaction) details

↓ Installation

```
npm install @tetherto/wdk-protocol-fiat-transak
```

Quick Start

Basic Usage

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

const widgetUrl = async (url) => {
  const response = await fetch('https://your-backend.example.com/transak/widget-url', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ url })
  })
}
```

```

    if (!response.ok) {
      throw new Error(`Failed to create Transak widget URL: ${response.status} ${response.statusText}`)
    }

    const { widgetUrl } = await response.json()

    return widgetUrl
  }

  // Initialize protocol without a wallet account.
  // You then pass `recipient`/`refundAddress` explicitly on buy/sell (see below).
  const transak = new TransakProtocol(undefined, {
    apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
    widgetUrl,
    environment: 'STAGING'
  })

  // Get a buy quote (cryptoAsset, fiatCurrency, fiatAmount, paymentMethod and network are all required)
  const buyQuote = await transak.quoteBuy({
    cryptoAsset: 'ETH',
    fiatCurrency: 'USD',
    fiatAmount: 100, // 100 USD, as a decimal in standard units
    paymentMethod: 'credit_debit_card',
    network: 'ethereum'
  })

  // Generate a buy widget URL
  const { buyUrl } = await transak.buy({
    fiatCurrency: 'USD',
    cryptoAsset: 'ETH',
    fiatAmount: 100,
    recipient: '0xabc'
  })

  console.log('Buy URL:', buyUrl)

  // Get a sell quote (cryptoAsset, fiatCurrency, cryptoAmount, paymentMethod and network are all required)
  const sellQuote = await transak.quoteSell({
    cryptoAsset: 'ETH',
    fiatCurrency: 'USD',
    cryptoAmount: 0.1, // 0.1 ETH, as a decimal in standard units
    paymentMethod: 'sepa_bank_transfer',
    network: 'ethereum'
  })

  // Generate a sell widget URL
  const { sellUrl } = await transak.sell({
    fiatCurrency: 'USD',
    cryptoAsset: 'ETH',
    cryptoAmount: 0.1,
    refundAddress: '0xabc'
  })

```

Using a Wallet Account

The first constructor argument is an optional WDK wallet account. When you bind one, `buy` and `sell` automatically use the account's address, so you don't need to pass `recipient/refundAddress` on every call:

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

// `walletManager` is your app's WDK wallet manager (chain-specific).
// `getAccount()` returns an IWalletAccount exposing getAddress().
const account = await walletManager.getAccount(0)

const transak = new TransakProtocol(account, {
  apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
  widgetUrl,
  environment: 'STAGING'
})

// No `recipient` needed — the account's address is filled in automatically.
const { buyUrl } = await transak.buy({
  fiatCurrency: 'USD',
  cryptoAsset: 'ETH',
  fiatAmount: 100
})

// Likewise, no `refundAddress` needed for sell.
const { sellUrl } = await transak.sell({
  fiatCurrency: 'USD',
  cryptoAsset: 'ETH',
  cryptoAmount: 0.1
})

// An explicit recipient/refundAddress still wins over the account address:
const { buyUrl: toOther } = await transak.buy({
  fiatCurrency: 'USD',
  cryptoAsset: 'ETH',
  fiatAmount: 100,
  recipient: '0xSomeOtherAddress'
})
```

Choosing an account type

The `account` argument accepts three forms:

- **undefined** — no wallet is bound. Pass `recipient/refundAddress` explicitly on `buy/sell`, or omit them to let the Transak widget prompt the user for the address. Ideal for quotes, backend/server usage, or when the destination address comes from elsewhere.
- **IWalletAccountReadOnly** — a read-only account. Its `getAddress()` is used to pre-fill the wallet address on `buy/sell`.
- **IWalletAccount** — a full wallet account. Works the same as a read-only account here, since the `buy/sell` flow never signs transactions (Transak's widget handles the crypto side).

Only `buy` and `sell` use the account, and only to fill in the wallet address when you don't pass one. `quoteBuy`, `quoteSell`, `getTransactionDetail`, and the `getSupported*` methods don't use it, so `undefined` is fine for those.

Amounts & Units

Inputs — `fiatAmount` and `cryptoAmount` are passed as **decimals in standard units**, matching Transak's API (not WDK base units):

- `fiatAmount` — e.g. `100` or `100.5` for 100.50 USD.
- `cryptoAmount` — e.g. `0.1` for 0.1 ETH.

Quote outputs — the `FiatQuote` returned by `quoteBuy/quoteSell` follows the WDK convention and reports amounts in **base units** as `bigint`:

- `cryptoAmount` / `fiatAmount` / `fee` are `bigint` in the asset's base unit (e.g. wei) and the currency's smallest unit (e.g. cents).
- `rate` is a decimal string (standard units).

So: you pass plain decimals in, and get `bigint` base units back. The module uses each asset's `decimals` and each currency's `roundOff` (from the supported-currencies lists) to do the conversion.

Values & Conventions

Pass `cryptoAsset`, `fiatCurrency`, `network`, and `paymentMethod` **exactly** as Transak spells them — the module matches them case-sensitively and won't fix the casing for you. Get it wrong and you'll see `Cannot find info for cryptoAsset and fiatCurrency`.

```
// ✓ Correct - values match Transak's conventions exactly
await transak.quoteBuy({
  cryptoAsset: 'USDT',           // upper-case symbol
  fiatCurrency: 'USD',          // upper-case ISO 4217 code
  network: 'ethereum',          // lower-case network name
  paymentMethod: 'credit_debit_card', // lower-case identifier
  fiatAmount: 100
})

// ✗ Wrong - the casing isn't fixed up for you, so this throws:
// Cannot find info for cryptoAsset and fiatCurrency
await transak.quoteBuy({
  cryptoAsset: 'usdt',
  fiatCurrency: 'usd',
  network: 'Ethereum',
  paymentMethod: 'credit_debit_card',
  fiatAmount: 100
})
```

Field	Convention	Examples
<code>cryptoAsset</code>	Upper-case symbol	<code>ETH</code> , <code>USDT</code> , <code>USDC</code> , <code>BTC</code> , <code>SOL</code> , <code>BNB</code> , <code>XRP</code> , <code>TRX</code> , <code>DOGE</code> , <code>LTC</code>

<code>fiatCurrency</code>	Upper-case ISO 4217 code	<code>USD, EUR, GBP, CHF, SEK, PLN, NOK, DKK</code>
<code>network</code>	Lower-case network name	<code>ethereum, tron, solana, bsc, polygon, mainnet</code> (BTC/LTC/XRP/DOGE)
<code>paymentMethod</code>	Lower-case identifier	<code>credit_debit_card, sepa_bank_transfer, gbp_bank_transfer, apple_pay, google_pay, pm_open_banking, pm_wire</code>

These are just examples. The real list depends on your Transak account and the user's country, so fetch it at runtime (see below) instead of hard-coding values.

Finding the exact values

Call `getSupportedCryptoAssets()` and `getSupportedFiatCurrencies()` to get the exact values. Each crypto asset gives you its `code` (the `cryptoAsset` symbol) and `networkCode` (the `network`):

```
const assets = await transak.getSupportedCryptoAssets()
// [
//   { code: 'ETH', networkCode: 'ethereum', decimals: 18, name: 'Ethereum', metadata: { ... } },
//   { code: 'USDT', networkCode: 'ethereum', decimals: 6, name: 'Tether USD', metadata: { ... } },
//   { code: 'USDT', networkCode: 'tron', decimals: 6, name: 'Tether USD', metadata: { ... } },
//   ...
// ]

const currencies = await transak.getSupportedFiatCurrencies()
// [
//   { code: 'USD', decimals: 2, name: 'US Dollar', metadata: { ... } },
//   { code: 'EUR', decimals: 2, name: 'Euro', metadata: { ... } },
//   ...
// ]
```

Payment methods come from each fiat currency's `metadata.paymentOptions` array — use the `id` of the option you want:

```
const [usd] = (await transak.getSupportedFiatCurrencies()).filter(c => c.code === 'USD')
const paymentMethods = usd.metadata.paymentOptions.map(o => o.id)
// e.g. ['credit_debit_card', 'apple_pay', 'google_pay', 'pm_wire', ...]
```

Assets on multiple networks

Transak identifies an asset by symbol **and** network, so a symbol like `USDT` can exist on several chains (`ethereum, tron, solana, ...`). Pass the `network` to pick the right one:

- `quoteBuy` / `quoteSell` — `network` is **required**.
- `buy` / `sell` — `network` is optional in `config`; when omitted the module uses the **first** matching entry from the supported-crypto list. Pass `config.network` to disambiguate:

```
await transak.buy({
  cryptoAsset: 'USDT',
  fiatCurrency: 'USD',
  fiatAmount: 100,
  config: { network: 'tron' }
})
```

API Reference

TransakProtocol

Main class for Transak integration.

Constructor

```
new TransakProtocol(account, config)
```

Parameters:

- `account` (`IWalletAccount` | `IWalletAccountReadOnly` | `undefined`): The wallet account to bind to the protocol. Used only by `buy/sell` to auto-fill the wallet address when `recipient/refundAddress` is omitted. Pass `undefined` for an unbound instance. See Choosing an account type.
- `config` (object): The protocol config
 - `apiKey` (string): Your Transak partner API key.
 - `widgetUrl` (function, optional): A callback that takes the generated URL and returns a session-based Transak widget URL. Implement it on your backend by calling Transak's Create Widget URL API (which needs your API secret). If omitted, the plain query-parameter URL is returned.
 - `cacheTime` (number, optional): The duration in milliseconds to cache supported currencies.
 - `environment` ("PRODUCTION" | "STAGING", optional): The environment to use for Transak endpoints and widget URLs. Defaults to "PRODUCTION". Use "PRODUCTION" for live transactions and "STAGING" for testing with non-real funds.

Methods

Method	Description	Returns
<code>buy(options)</code>	Generates a widget URL to purchase crypto	<code>Promise<BuyResult></code>

<code>sell(options)</code>	Generates a widget URL to sell crypto	<code>Promise<SellResult></code>
<code>quoteBuy(config)</code>	Gets a quote for a crypto asset purchase	<code>Promise<TransakBuyQuote></code>
<code>quoteSell(config)</code>	Gets a quote for a crypto asset sale	<code>Promise<TransakSellQuote></code>
<code>getTransactionDetail(txId)</code>	Retrieves the details of an order	<code>Promise<TransakTransactionDetail></code>
<code>getSupportedCryptoAssets()</code>	Retrieves a list of supported crypto assets	<code>Promise<TransakSupportedCryptoAsset[]></code>
<code>getSupportedFiatCurrencies()</code>	Retrieves a list of supported fiat currencies	<code>Promise<TransakSupportedFiatCurrency[]></code>
<code>getSupportedCountries()</code>	Retrieves a list of supported countries	<code>Promise<TransakSupportedCountry[]></code>

`buy(options)`

Generates a widget URL to purchase crypto.

Options:

- `fiatCurrency` (string): The fiat currency code (e.g., 'USD').
- `cryptoAsset` (string): The crypto asset code (e.g., 'ETH').
- `fiatAmount` (number, optional): The fiat amount, as a decimal in standard units (e.g. `100.5`).
- `cryptoAmount` (number, optional): The crypto amount, as a decimal in standard units (e.g. `0.1`).
- `recipient` (string, optional): The wallet address to receive funds. If not provided, uses the account address.
- `config` (object, optional): Additional Transak widget parameters (including `network`).

`sell(options)`

Generates a widget URL to sell crypto.

Options:

- `fiatCurrency` (string): The fiat currency code (e.g., 'USD').
- `cryptoAsset` (string): The crypto asset code (e.g., 'ETH').
- `fiatAmount` (number, optional): The fiat amount, as a decimal in standard units (e.g. `100.5`).
- `cryptoAmount` (number, optional): The crypto amount, as a decimal in standard units (e.g. `0.1`).

- `refundAddress` (string, optional): The wallet address for refunds. If not provided, uses the account address.
- `config` (object, optional): Additional Transak widget parameters (including `network`).

`quoteBuy(config)`

Gets a quote for a crypto asset purchase. Takes a single `TransakQuoteBuyParams` object (not an options wrapper):

- `cryptoAsset` (string, **required**): The crypto asset code (e.g. `'ETH'`).
- `fiatCurrency` (string, **required**): The fiat currency code (e.g. `'USD'`).
- `fiatAmount` (number, **required**): The fiat amount, as a decimal in standard units (e.g. `100.5`).
- `paymentMethod` (string, **required**): The payment method to price the quote against (e.g. `'credit_debit_card'`).
- `network` (string, **required**): The network of the crypto currency (e.g. `'ethereum'`).

`quoteSell(config)`

Gets a quote for a crypto asset sale. Takes a single `TransakQuoteSellParams` object (not an options wrapper):

- `cryptoAsset` (string, **required**): The crypto asset code (e.g. `'ETH'`).
- `fiatCurrency` (string, **required**): The fiat currency code (e.g. `'USD'`).
- `cryptoAmount` (number, **required**): The crypto amount, as a decimal in standard units (e.g. `0.1`).
- `paymentMethod` (string, **required**): The payout method to price the quote against (e.g. `'sepa_bank_transfer'`).
- `network` (string, **required**): The network of the crypto currency (e.g. `'ethereum'`).

`getTransactionDetail(txId)`

Retrieves the details of an order.

Parameters:

- `txId` (string): The Transak order ID.

`getSupportedCryptoAssets()`

Retrieves a list of supported crypto assets. Each entry: `{ code, networkCode, decimals, name, metadata }`, where `code` is the `cryptoAsset` symbol and `networkCode` is the `network` — use both exactly as returned (see Values & Conventions). The full Transak object is under `metadata`.

`getSupportedFiatCurrencies()`

Retrieves a list of supported fiat currencies. Each entry: `{ code, decimals, name, metadata }`, where `code` is the `fiatCurrency` code. Available `paymentMethod` identifiers are found in `metadata.paymentOptions[].id`.

`getSupportedCountries()`

Retrieves a list of supported countries. Each entry: { `code`, `isBuyAllowed`, `isSellAllowed`, `name`, `metadata` }.

Notes

- Works with the networks and currencies your Transak account supports.
- This package covers the basics. For the full list of widget parameters, see the [Transak query parameters docs](#).
- Get your `apiKey` from the [Transak Partner dashboard](#).
- Test the full buy/sell flow in the `STAGING` environment before going live.

Security Considerations

- Keep your Transak API secret on your backend, never in client code. Expose a small backend endpoint that calls Transak's Create Widget URL API, and have your `widgetUrl` callback call that endpoint. See [Transak's migration to API-based widget URLs](#).

Development

Building

```
# Install dependencies
npm install

# Build TypeScript definitions
npm run build:types

# Lint code
npm run lint

# Fix linting issues
npm run lint:fix
```

Testing

```
# Run tests
npm test

# Run tests with coverage
npm run test:coverage
```

License

This project is licensed under the Apache License 2.0 - see the LICENSE file for details.

Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

Support

For support, please open an issue on the GitHub repository.